

Sistema IoT para Estacionamiento tipo Carrusel — "Smart Carousel Parking"



Asignatura / Sector:

IoT Industrial

Ciudades inteligentes y movilidad urbana

Integrantes:

Enzo Fernández

Lucas González

Índice

Índice.....	2
1. Descripción del problema y contexto.....	3
2. Objetivos de la solución.....	4
3. Justificación de IoT.....	4
4. Arquitectura IoT.....	5
5. Selección y justificación tecnológica.....	6
6. Flujo de datos.....	8
7. Análisis de Seguridad.....	10
8. Reflexión ética.....	11
9. Caso de negocio.....	12
10. Evidencias del prototipo.....	14
11. Conclusiones.....	15

1. Descripción del problema y contexto

Estacionar es uno de los cuellos de botella menos visibles, pero más costosos, de la movilidad urbana. Distintos estudios cuantifican el problema: según INRIX, un conductor promedio en Estados Unidos pierde alrededor de 17 horas al año buscando estacionamiento, lo que equivale a unos USD 345 anuales por conductor en tiempo, combustible y emisiones, y a cerca de USD 72.700 millones por año a nivel país. En grandes ciudades la cifra se dispara: en Nueva York el promedio supera las 100 horas anuales por conductor. A esto se suma que, en zonas céntricas congestionadas con cupos mal aprovechados, los estudios clásicos de Donald Shoup encontraron que en promedio cerca del 30 % del tránsito corresponde a vehículos "dando vueltas" buscando lugar (*cruising for parking*), con un rango muy amplio según la ciudad. Ese tránsito improductivo agrava la congestión, eleva las emisiones y desgasta la infraestructura.

Frente a la escasez de suelo urbano, el estacionamiento tipo carrusel aparece como una respuesta atractiva: es una estructura rotatoria vertical que almacena varios vehículos en una superficie mínima, ideal para ciudades densas donde el metro cuadrado es caro y escaso. Sin embargo, este tipo de instalación arrastra dos problemas operativos que hoy no están resueltos:

- a) Falta de visibilidad de la disponibilidad en tiempo real. El conductor no tiene forma de saber, antes de llegar, si el carrusel más cercano tiene una posición libre. Termina trasladándose hasta el lugar para descubrir que está completo, reproduciendo exactamente el *cruising* y la congestión que la solución pretendía evitar. Del lado del operador o del municipio, tampoco existe un panel que muestre el estado de todos los carruseles de la ciudad de un vistazo.
- b) Dependencia de un único sistema electromecánico crítico. A diferencia de una playa convencional, el carrusel solo entrega o retira un vehículo si su motor y mecanismo rotativo funcionan. Si el motor falla, *toda* la estructura queda fuera de servicio y los vehículos almacenados quedan retenidos hasta la reparación. Hoy el mantenimiento de estos equipos suele ser correctivo (se interviene cuando la falla ya ocurrió) o preventivo por calendario, y no se basa en la condición real del equipo —temperatura, vibración y consumo del motor—, que es justamente lo que anticipa una avería.

Contexto del caso. El proyecto se sitúa en un entorno urbano (se usa Asunción como referencia en los *topics*), con un operador o municipio que administra varios carruseles distribuidos en la ciudad. La mecánica de cada carrusel ya está gobernada por un PLC (mundo OT) que no se modifica: el proyecto se concentra en la capa IoT que lee esas señales de forma no intrusiva y las pone en valor.

El problema, en una frase: no existe hoy visibilidad en tiempo real de la ocupación por posición, ni un monitoreo de la condición del motor que permita anticipar fallas, lo que se traduce en tiempo perdido y congestión para el conductor y en paradas no planificadas y costos para el operador.

2. Objetivos de la solución

Objetivo general.

Diseñar e implementar un prototipo (simulado en Wokwi) de un sistema IoT de cuatro capas que monitoree en tiempo real la ocupación de los slots y la salud del motor de estacionamientos tipo carrusel.

Objetivos específicos.

1. Leer de forma no intrusiva (solo lectura) las señales que el PLC ya gestiona (estado de cada una de las 8 posiciones por carrusel) y publicarlas por MQTT en tiempo real.
2. Monitorear las variables del motor (temperatura, vibración y consumo) y clasificar su condición en tres niveles (NORMAL / ADVERTENCIA / FALLA) como base de un esquema de mantenimiento predictivo.
3. Diseñar la arquitectura IoT de 4 capas con convergencia IT/OT, usando el ESP32 como *gateway* de borde entre el mundo OT (PLC) y el mundo IT/IoT.
4. Implementar el pipeline de datos de extremo a extremo: ESP32 → MQTT → broker Mosquitto → servidor Node.js → InfluxDB → frontends, con REST + WebSocket para la actualización en vivo.
5. Construir los dos productos: un dashboard de control (estado de los 3 carruseles, mapa de slots, KPIs y alertas de mantenimiento) y una web app móvil para conductores con la disponibilidad cercana.
6. Calcular KPIs de ciudad inteligente y de mantenimiento: porcentaje de ocupación, disponibilidad/*uptime* del carrusel, y MTBF / MTTR del motor.
7. Incorporar seguridad (matriz STRIDE y MQTT sobre TLS) y una reflexión ética sobre los datos que el sistema recolecta.

3. Justificación de IoT

El problema planteado tiene tres características que, juntas, descartan una solución manual o aislada y apuntan directamente a una arquitectura IoT: requiere datos continuos provenientes de sensores distribuidos (cada slot y cada motor, en varios carruseles repartidos por la ciudad), exige baja latencia (el estado "libre/ocupado" pierde valor si llega tarde al conductor) y necesita visualización remota simultánea

para dos audiencias distintas. Una planilla, una inspección periódica o un tablero local en cada carrusel no resuelven ninguno de los tres puntos: no son en tiempo real, no son remotos y no escalan a una red de carruseles.

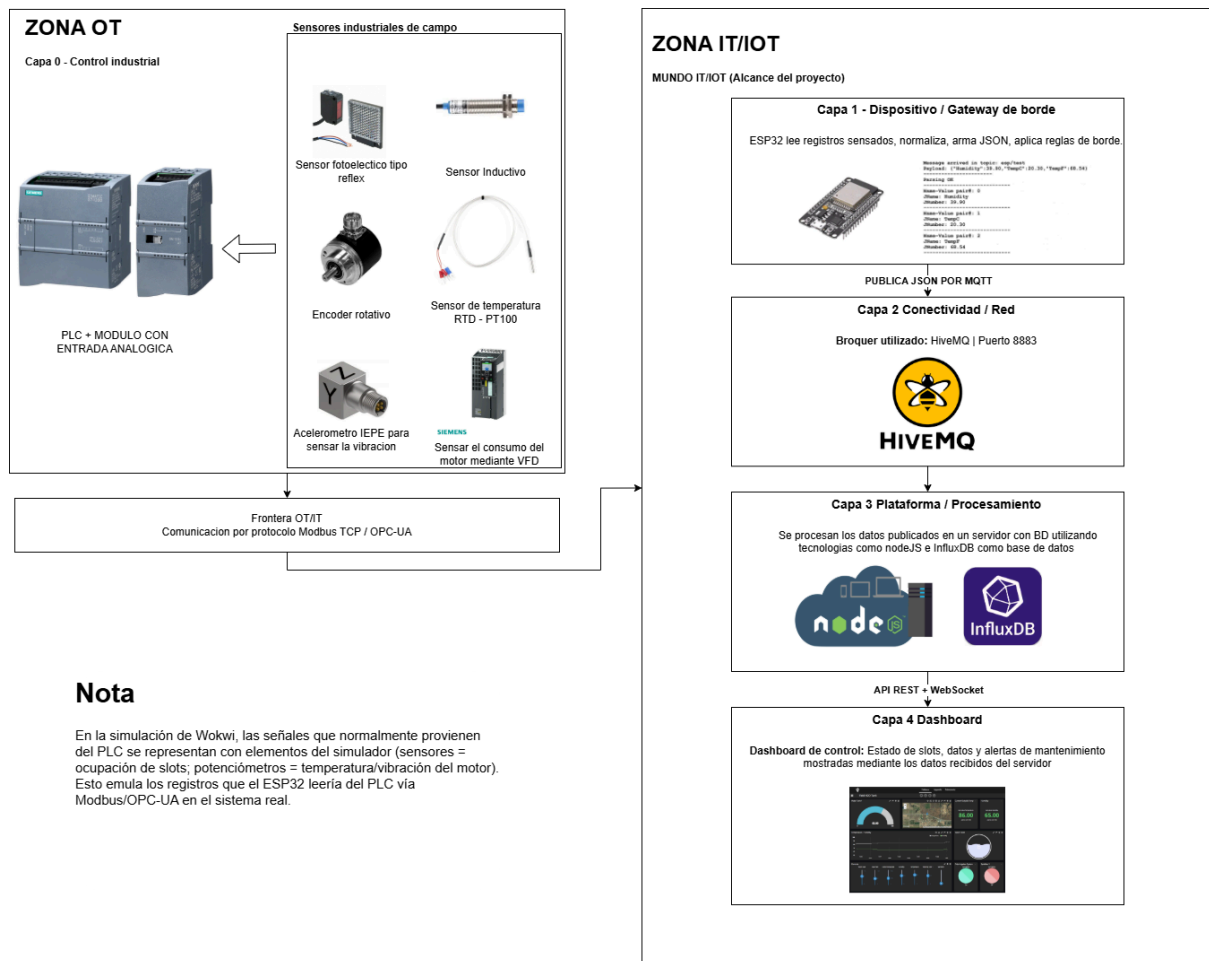
Una arquitectura IoT de cuatro capas encaja exactamente con esas necesidades:

- Capa de dispositivo / borde. El ESP32 actúa como *gateway*: lee las señales del PLC, normaliza los valores, arma el JSON y aplica reglas simples en el borde (por ejemplo, marcar una alerta de motor) antes de transmitir. Procesar en el borde reduce el tráfico y la latencia.
- Capa de conectividad / red. El nodo publica por MQTT, un protocolo *pub/sub* ligero y estándar de facto en IoT, ideal para muchos sensores que emiten mensajes pequeños y frecuentes hacia un broker Mosquitto.
- Capa de plataforma / procesamiento. Un servidor Node.js se suscribe a los *topics*, calcula slots libres y alertas, persiste el histórico en InfluxDB (una base de series temporales, natural para telemetría con marca de tiempo) y sirve los datos por REST + WebSocket.
- Capa de aplicación. Los dos frontends —dashboard del operador y web app del conductor— consumen esa información y la convierten en decisiones: dónde estacionar y cuándo intervenir un motor.

A esto se suma el concepto de convergencia IT/OT (Semana 4), que es el corazón del diseño: la mecánica del carrusel pertenece al mundo OT y la gobierna un PLC que no se toca. El ESP32 hace de puente hacia el mundo IT/IoT, leyendo los registros del PLC por Modbus TCP / OPC-UA solo en modo lectura. Así se obtiene el dato sin alterar ni poner en riesgo el control industrial —un patrón realista y seguro, muy distinto de "cablear sensores nuevos a un microcontrolador".

Finalmente, el caso pertenece de lleno a la vertical de ciudades inteligentes y movilidad urbana (Semana 5): el mismo dato de ocupación que ayuda al conductor a evitar el *cruising* le sirve al municipio para medir uso, planificar y reducir congestión y emisiones. El monitoreo del motor aporta el ángulo industrial (IIoT y mantenimiento predictivo). En conjunto —tiempo real, sensores distribuidos, procesamiento en plataforma y doble visualización remota— el problema cae justo en el terreno donde IoT aporta más valor que cualquier alternativa.

4. Arquitectura IoT



5. Selección y justificación tecnológica

La solución se divide en dos mundos: los sensores industriales de campo, que pertenecen a la capa OT y ya están gobernados por el PLC del carrusel (el proyecto no los instala, solo justifica su uso para describir la solución de extremo a extremo), y la capa IoT, que es lo que efectivamente construye este trabajo. El ESP32 actúa como puente entre ambos: lee las señales del PLC de forma no intrusiva y las lleva al mundo IT/IoT.

Sensores industriales de campo (capa OT — del PLC)

Son componentes de grado industrial, los que realmente usa un estacionamiento automatizado. Se documentan para justificar la solución completa, aunque el ESP32 solo lee sus valores a través del PLC.

Variable	Componente industrial	Justificación
Ocupación de slot	Sensor inductivo de proximidad (PNP 24 V)	Estándar en parking y control de accesos; sin contacto, robusto, resiste intemperie y el paso de vehículos. No es un sensor de hobby.
Posición del carrusel	Barrera fotoeléctrica PNP 24V Encoder rotativo industrial (incremental/absoluto)	Determina en qué posición está cada slot con alta precisión mediante el control digital de la barrera fotoeléctrica y el feedback del encoder.
Temperatura del motor	PT100 / RTD	Precisión industrial para el monitoreo de condición (mantenimiento predictivo).
Vibración del motor	Acelerómetro IEPE / sensor de vibración industrial	Detecta desgaste de rodamientos antes de la falla (norma ISO 10816).
Consumo del motor	Transformador de corriente (CT)	Detecta sobreesfuerzo y anomalías de carga del motor.

Capa IoT (lo que construye el proyecto)

Componente	Elección	Justificación
Gateway de borde	ESP32	Wi-Fi/Ethernet integrados, soporta Modbus TCP y MQTT, bajo costo y usado en el curso. Cumple el rol de gateway IT/OT y aplica reglas simples en el borde (clasificación del motor).
Integración con el PLC	Modbus TCP / OPC-UA (solo lectura)	Permite leer los registros del PLC sin alterar el control industrial. Es el patrón de convergencia IT/OT (Semana 4): el dato se obtiene sin poner en riesgo la operación.
Protocolo IoT	MQTT	Protocolo pub/sub ligero, estándar de facto en IoT, ideal para muchos nodos que envían mensajes pequeños y frecuentes.
Seguridad del transporte	MQTT sobre TLS (puerto 8883)	Cifra la telemetría en tránsito y autentica el nodo con usuario/contraseña; mitiga tampering e info disclosure (ver análisis STRIDE).

Broker	HiveMQ Cloud	Broker MQTT gestionado, con TLS y autenticación incluidos, sin necesidad de mantener infraestructura propia de mensajería; escala a múltiples carruseles.
Servidor de aplicación	Node.js en Raspberry Pi	Servidor de borde de bajo consumo que se suscribe a los topics, calcula slots libres y alertas, persiste los datos y expone la API. Node.js encaja con la actualización en tiempo real (WebSocket).
Base de datos	InfluxDB	Base de series temporales, natural para telemetría con marca de tiempo; permite el histórico de ocupación y del motor y las consultas para los gráficos.
Exposición de servicios	REST + WebSocket (túnel Cloudflare)	REST para consultas puntuales e históricos; WebSocket para empujar cada dato nuevo al dashboard sin recargar. El túnel publica el servidor local de forma segura.
Visualización	Dashboard web (HTML/JS + Chart.js)	Panel de control del operador: estado de los 3 carruseles, grilla de slots, salud del motor, gráficos temporales y alertas, accesible desde cualquier navegador.
Red de acceso	Wi-Fi / Ethernet industrial	Suficiente para nodos fijos dentro de la estructura del carrusel, con energía disponible. Para un despliegue urbano disperso se evaluaría NB-IoT frente a LoRaWAN. NB-IoT ofrece mayor ancho de banda y soporte de IP/MQTT que LoRaWAN, a cambio de mayor consumo —aceptable porque los nodos no son a batería.

Las decisiones siguen el flujo natural de una arquitectura IoT de cuatro capas y priorizan tres criterios: no intrusión sobre el OT (lectura del PLC por Modbus/OPC-UA, sin tocar el control), bajo costo y mantenimiento (ESP32 + Raspberry Pi + broker gestionado, todo open source o de capa gratuita) y tiempo real (MQTT + WebSocket para que el estado "libre/ocupado" y las alertas del motor lleguen sin demora). El uso de un broker gestionado (HiveMQ) con TLS resuelve la seguridad del transporte sin sumar carga operativa, mientras que InfluxDB aporta el histórico necesario para los KPIs de ocupación y para el mantenimiento predictivo del motor.

6. Flujo de datos

Los datos recorren cinco etapas desde el sensor físico hasta el usuario final.

Origen en la capa OT.

Los sensores industriales instalados en el carrusel —detectores inductivos de slot, encoder de posición, PT100 de temperatura del motor, acelerómetro IEPE y transformador de corriente— están conectados al PLC que gobierna la mecánica. El ESP32 actúa como gateway de borde y lee los registros del PLC en modo solo lectura mediante **Modbus TCP / OPC-UA**, sin interferir en el control industrial. En la simulación de Wokwi, pulsadores y potenciómetros representan esas señales para demostrar el concepto sin alterar el diseño industrial.

Normalización y publicación en el borde.

El ESP32 convierte los valores crudos, aplica reglas simples (ej.: si temperatura > umbral → **alerta: true**) y publica un JSON estructurado por **MQTT** sobre TLS (puerto 8883) hacia el broker **HiveMQ Cloud** con autenticación usuario/contraseña. Se utilizan tres topics jerárquicos, uno por carrusel:

parking/asuncion/carrusel-01/telemetry

parking/asuncion/carrusel-02/telemetry

parking/asuncion/carrusel-03/telemetry

La frecuencia de publicación es **on-change + heartbeat cada 10 s** para el estado de los slots (QoS 1, dato crítico), y **cada 5 s** para la telemetría del motor (QoS 0, alta frecuencia). Cada carrusel publica además un topic de estado **.../status** con Last Will Testament para detectar desconexiones.

Ejemplo de payload JSON:

```
{
  "carrusel": "carrusel-01",
  "ts": "2026-06-01T14:32:10Z",
  "slots": {
    "1": "ocupado", "2": "libre", "3": "ocupado",
    "4": "libre", "5": "libre", "6": "ocupado",
    "7": "libre", "8": "ocupado"
  },
  "libres": 4,
  "motor": { "temp_c": 41.2, "vibracion": 0.07, "corriente_a": 3.4 },
  "alerta": false
}
```

Procesamiento en el servidor.

El servidor Node.js (Express + mqtt.js + ws, corriendo en Raspberry Pi) se suscribe al broker, parsea cada mensaje, calcula el total de slots libres y clasifica la condición del motor en tres niveles —**NORMAL / ADVERTENCIA / FALLA**— comparando temperatura y vibración contra umbrales configurables. Los datos se persisten en **InfluxDB** usando el timestamp del servidor (no el del ESP32, que en Wokwi no dispone de RTC). Esto construye el histórico de series temporales necesario para el análisis de patrones de ocupación y la detección de tendencias en el motor.

Exposición a los frontends.

El servidor expone los datos por dos canales: una **API REST** (`GET /iot/api/carruseles`, `/disponibilidad`, `/historial/:id`, `/alertas`) para consultas puntuales, y un **WebSocket** en `/iot/ws` que hace broadcast de cada mensaje MQTT recibido hacia todos los clientes conectados, garantizando actualización en tiempo real sin polling.

Consumo en la capa de aplicación.

El dashboard del operador muestra el mapa de los 3 carruseles, el estado slot a slot, los datos de disponibilidad y las alertas de mantenimiento.

El recorrido completo queda entonces: **Sensores** → **PLC (OT)** → **[Modbus TCP, solo lectura]** → **ESP32 gateway** → **Wi-Fi/TLS** → **HiveMQ Cloud (MQTT)** → **Node.js** → **InfluxDB** → **REST + WebSocket** → **Dashboard / Web app**

7. Análisis de Seguridad

Autenticación de dispositivos

El sistema implementa autenticación en todos los puntos de comunicación entre dispositivos. El ESP32, actuando como gateway de borde, se conecta al broker HiveMQ Cloud mediante credenciales únicas de usuario y contraseña, lo que garantiza que únicamente los dispositivos autorizados del proyecto puedan publicar mensajes en los topics del sistema. Esta decisión responde directamente a la amenaza de **Spoofing** del modelo STRIDE, ya que impide que dispositivos no identificados inyecten datos falsos en la plataforma.

Del mismo modo, el servidor Node.js se autentica ante el broker utilizando las mismas credenciales a través de una conexión `mqtt://`, asegurando que el

consumidor de los mensajes también esté debidamente identificado. Las credenciales del servidor se gestionan mediante variables de entorno definidas en el contenedor docker del servidor, separando la configuración sensible del código fuente de la aplicación y siguiendo buenas prácticas de gestión de secretos. Esta arquitectura cubre la amenaza de Information Disclosure, evitando que datos sensibles queden expuestos en el código de la aplicación.

Cifrado de comunicaciones

Todas las comunicaciones externas del sistema se encuentran cifradas. El ESP32 publica sus mensajes MQTT exclusivamente sobre TLS en el puerto 8883, garantizando la confidencialidad e integridad de la telemetría en tránsito desde el dispositivo hasta el broker en la nube. Esta medida mitiga directamente la amenaza de **Tampering**, ya que ningún actor externo puede interceptar ni modificar los mensajes publicados por el gateway. La elección del puerto 8883 sobre el estándar 1883 fue una decisión deliberada para cumplir con los requisitos de seguridad de la consigna.

La comunicación entre el servidor y el broker replica el mismo nivel de protección mediante mqtt://, manteniendo cifrado extremo a extremo en toda la cadena de mensajería. Por su parte, el acceso al dashboard desde el navegador del operador se realiza íntegramente sobre HTTPS gracias al túnel de Cloudflare, que actúa como capa de terminación TLS y protege los datos visualizados en tránsito. En conjunto, el sistema garantiza que ningún dato —ni de telemetría ni de visualización— viaja en texto plano por redes públicas, cubriendo las amenazas de **Tampering** e **Information Disclosure** en todos los tramos externos de la arquitectura.

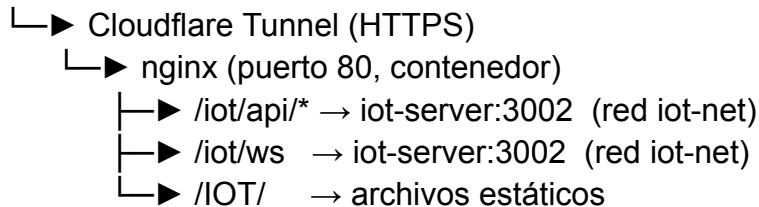
Segmentación de red

La arquitectura de red del sistema fue diseñada con un enfoque de mínima exposición. Los servicios internos —el servidor Node.js y la base de datos InfluxDB— se despliegan dentro de una red Docker privada (iot-net) sin puertos expuestos a internet, lo que garantiza que ninguno de estos componentes sea accesible directamente desde el exterior. Esta decisión mitiga amenazas de Information **Disclosure** al aislar la capa de datos y la lógica de negocio detrás de una red controlada.

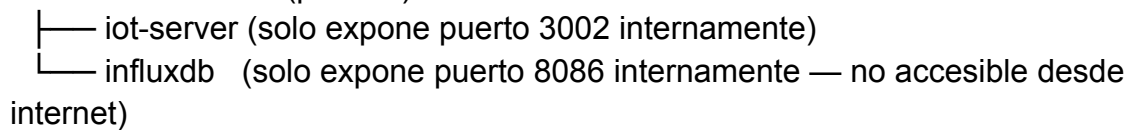
El único punto de entrada público al sistema es el servidor nginx, configurado como reverse proxy. Nginx actúa como frontera de seguridad, enrutando selectivamente solo las rutas habilitadas (/iot/api/* y /iot/ws) hacia el servidor Node.js, sin exponer ningún otro servicio interno. Por encima de nginx, el túnel de Cloudflare añade una capa adicional de segmentación al ocultar la dirección IP real de la Raspberry Pi, eliminando la posibilidad de ataques directos sobre la infraestructura física y

mitigando la amenaza de **Denial of Service** sobre el servidor. Esta arquitectura en capas —dispositivo, broker en la nube, red Docker privada, reverse proxy, túnel cifrado— representa una implementación sólida del principio de defensa en profundidad aplicado a sistemas IoT industriales.

Internet



Red Docker iot-net (privada):



Obs: El dominio utilizado para el acceso web fue generado mediante cloudflare con una suscripción gratuita (para realizar el prototipado de la solución) por lo que se genera un dominio aleatorio por cada vez que el servidor se reinicia.

8. Reflexión ética

El sistema recolecta dos tipos de datos: el estado de ocupación de cada slot (libre/ocupado) y la telemetría del motor (temperatura, vibración y corriente). Ninguno de los dos identifica a una persona ni a un vehículo: no se registran patentes, ni rostros, ni datos del conductor. Esto es una ventaja de diseño deliberado —minimización de datos—: el sistema cumple su objetivo (saber dónde hay lugar y cuidar el motor) sin recolectar información personal. Si en una etapa futura se agregara identificación de vehículos (por ejemplo, lectura de patente para facturación), esos datos sí serían personales y deberían tratarse con consentimiento, base legal y mayores recaudos.

El dashboard de control es para el operador o el municipio, por lo que su acceso debe estar protegido con autenticación y roles (operador vs. administrador), separando quién puede ver y quién puede configurar. La información de disponibilidad que llegaría a una eventual app del conductor es agregada (cuántos lugares libres hay), no individual, de modo que no expone el comportamiento de ninguna persona.

Aunque hoy el dato es anónimo, los patrones de ocupación a lo largo del tiempo podrían, si se cruzaran con otras fuentes, revelar rutinas de uso de una zona. El

principio ético es usar los datos solo para el fin declarado (gestión del estacionamiento y mantenimiento) y no para perfilar usuarios ni revenderlos. Conviene definir por contrato que el operador no haga uso secundario de los históricos.

La telemetría en crudo no necesita guardarse indefinidamente. Una política razonable es conservar el detalle reciente (por ejemplo, 30–90 días) para diagnóstico y mantenimiento, y a partir de ahí guardar solo agregados (promedios por hora/día) para estadística, descartando el dato fino. Así se reduce la superficie de riesgo sin perder valor analítico.

Las comunicaciones viajan cifradas con TLS (MQTT 8883) y el nodo se autentican con credenciales, lo que evita que un tercero lea o falsee los datos en tránsito. Sumado a la minimización de datos y al control de acceso del dashboard, el sistema protege la privacidad por diseño, no como un agregado posterior.

El impacto esperado es positivo: al evitar que los conductores circulen buscando lugar, se reduce la congestión y, con ella, el consumo de combustible y las emisiones en la zona. Además, optimizar el uso de estructuras verticales aprovecha mejor el escaso suelo urbano. El reverso ético a vigilar es la equidad en el acceso: la información debería ser pública y gratuita para el conductor, de modo que la mejora de movilidad beneficie a todos y no solo a quienes paguen un servicio premium.

9. Caso de negocio

Estimación básica para un sitio con 3 carruseles (24 posiciones). Se evalúa únicamente la capa IoT que agrega el proyecto; la estructura mecánica del carrusel y su PLC ya existen y quedan fuera del cálculo. Los valores son aproximados, con fines de dimensionamiento, en dólares (USD).

CAPEX — inversión inicial (una sola vez)

Ítem	Detalle	Costo (USD)
Gateways de borde	3 × ESP32 + caja IP67 + cableado/optoacopladores (1 por carrusel)	135
Servidor local	Raspberry Pi 4 + almacenamiento SSD + UPS	150
Red	Punto de acceso / switch industrial del sitio	80

Desarrollo de software	Servidor (Node.js + InfluxDB) y dashboard web — desarrollo único	1.200
Instalación y puesta en marcha	Integración con el PLC (Modbus/OPC-UA) y mano de obra	300
Total CAPEX		≈ 1.865

El desarrollo de software es un costo único que se amortiza si la solución se replica en más sitios (el mismo servidor y dashboard sirven para varios carruseles).

OPEX — costos operativos (anuales)

Ítem	Detalle	Costo (USD/año)
Conectividad	Internet / enlace 4G del sitio	240
Broker MQTT	HiveMQ Cloud (capa gratuita en este alcance; escalable a plan pago)	0
Energía	Consumo de ESP32 + Raspberry Pi (muy bajo)	15
Mantenimiento de software	Actualizaciones, monitoreo, túnel de acceso	360
Total OPEX		≈ 615 (≈ 51/mes)

Retorno de inversión (ROI)

Fuente de valor	Estimación anual (USD)
Mayor rotación de vehículos (menos tiempo de búsqueda → más autos atendidos)	≈ 9.000
Mantenimiento predictivo (evita 1–2 paradas no planificadas del carrusel/año)	≈ 1.500
Beneficio anual estimado	≈ 10.500

Con un beneficio anual estimado de ~10.500 USD frente a una inversión inicial de ~1.865 USD y ~615 USD/año de operación, el período de recuperación es inferior a 3 meses y el ROI del primer año supera el 400 %. Aun con supuestos

conservadores (la mitad del beneficio), la inversión se recupera dentro del primer año.

Impacto operativo, económico y social

- **Operativo:** el operador pasa de un mantenimiento correctivo/por calendario a uno basado en condición, reduciendo paradas imprevistas y aumentando la disponibilidad del carrusel.
- **Económico:** mayor rotación y menos tiempo muerto se traducen en más ingresos por la misma infraestructura; el costo marginal de sumar carruseles al sistema es bajo.
- **Social y ambiental:** menos vehículos "dando vueltas" buscando lugar implica menos congestión y menos emisiones en la zona, además de una mejor experiencia para el conductor.

KPIs de seguimiento

KPI	Qué mide	Meta orientativa
% de ocupación	Uso real de cada carrusel	Optimizar 60–85 % (ni ocioso ni saturado)
Disponibilidad (uptime)	% de tiempo operativo del carrusel	> 98 %
MTBF / MTTR	Tiempo entre fallas / tiempo de reparación del motor	MTBF ↑ · MTTR ↓
Reducción del tiempo de búsqueda	Minutos que ahorra el conductor	– 30 % vs. sin sistema
Reducción de emisiones	CO ₂ evitado por menor cruising	Tendencia decreciente

10. Evidencias del prototipo

La simulación fue realizada por medio de la manipulación desde el entorno de Wokwi y la observación del comportamiento en el dashboard desarrollado, se adjunta la grabacion de pantalla realizada para la captura de la simulación.

 Grabación Simulacion.mp4

11. Conclusiones

El proyecto resolvió un problema concreto de la movilidad urbana: la falta de visibilidad en tiempo real de los lugares libres en estacionamientos tipo carrusel y la ausencia de un monitoreo del motor que anticipe fallas.

La solución, demostrada con un prototipo, lo resuelve con una arquitectura IoT de cuatro capas y convergencia IT/OT: el ESP32 lee de forma no intrusiva las señales del PLC, clasifica el estado del motor y publica por MQTT/TLS a HiveMQ; un servidor Node.js en Raspberry Pi persiste en InfluxDB y alimenta por REST + WebSocket un dashboard en tiempo real. El prototipo en Wokwi validó el flujo completo (lectura, publicación, almacenamiento, visualización y alertas) y el análisis de datos mostró los patrones de ocupación y la detección temprana de anomalías del motor.

El impacto es doble: para el conductor, menos tiempo de búsqueda y menos congestión y emisiones; para el operador, mayor disponibilidad del carrusel y un retorno de inversión rápido, todo con datos no personales (privacidad por diseño).

Como trabajo futuro queda la integración real al PLC por Modbus/OPC-UA, el uso del histórico real de InfluxDB, la app móvil para el conductor y la evaluación de NB-IoT para escalar a más carruseles. En síntesis, se logró una solución IoT coherente de punta a punta que responde directamente al problema y aporta valor al usuario y al operador.